

Software Engineering Concept (SE LAB05)

Topic: Concept of Multithreading

Lab Overview

The purpose of this lab is to deepen your understanding of multithreading and concurrency in Java through practical implementation of various concepts.

Task 1: File Processing with Multithreading

Create a Java program that reads multiple text files concurrently using threads. Each thread should:

Read a different text file

- Count the number of words, lines, and characters in the file
- Print the results with thread name and processing time. E.g.,
- Use at least 3 different files and 3 threads
- Ensure to limit the number of concurrent threads to avoid overwhelming the system

Requirements:

- Implement using both Thread class extension and Runnable interface
- Handle file not found exceptions properly
- Measure and display execution time for each thread
- Create a folder named "bunchOfFiles" with sample text files for testing (at least 5 files with varying sizes) and put some other file types to test exception handling
- The summary of results should be displayed after all threads complete

Step execution and approach:

- Running your code inside Task1.java, FileThread, and FileRunnable
- Create three text files and process them concurrently using multithreading.
- Use two threading techniques:
 - One class extends Thread (FileThread)
 - One class implements Runnable (FileRunnable)
- Each thread reads one file only, so file processing is done in parallel.
- For each file, the thread:
 - Reads the file line by line
 - Counts words, lines, and characters
- Use AtomicInteger for total words, lines, and characters:
 - Ensures thread-safe updates
 - Prevents race conditions when multiple threads update shared totals
- Start all threads at the same time and wait for them using join().
- Measure total execution time from start to finish.
- Display:

- ◦ Per-thread processing results
- ◦ Overall totals after all threads complete

```
Scanning files in folder: bunchOfFiles...
----- RESULTS -----
Thread-2 processed file3.txt: 100 words, 1 lines, 745 characters in 9ms
Thread-1 processed file2.txt: 70 words, 1 lines, 519 characters in 9ms
Thread-0 processed file1.txt: 45 words, 1 lines, 296 characters in 10ms
-----
Total files processed: 3
Total word(s): 215
Total line(s): 3
Total character(s): 1560
Total processing time: 44 ms
```

Task 2: Producer-Consumer with Priority Queue

Implement a Producer-Consumer pattern with the following specifications:

- Use a priority queue as the shared buffer (capacity: 10)
- Producer generates random numbers (1-100) with random priorities
- Consumer processes items based on priority (highest first)
- Create 2 producer threads and 3 consumer threads
- Run for 30 seconds and display statistics

Requirements:

- Implement proper synchronization
- Display buffer state after each operation
- Show total items produced and consumed by each thread

Step execution and approach:

- This task run in Task2.java
- Use a shared PriorityBlockingQueue (capacity 10) as the buffer so producers and consumers can safely access it without manual synchronized blocks, and items are always ordered by priority (highest first).
- Producers continuously generate random items (value + priority) and insert them into the queue; if the queue is full, they automatically block until space is available.
- Consumers continuously take items from the queue; if the queue is empty, they automatically block until an item is produced.
- The Item class implements Comparable so the priority queue always delivers the highest-priority item first, not FIFO order.
- The program runs all producer and consumer threads for 30 seconds, then interrupts them, waits for completion, and prints how many items each thread produced or consumed.

Output:

```

Consumer-3 consumed: (Value: 62, Priority:27) | Buffer: [(Value: 58, Priority:24), (Value: 52, Priority:16), (Value: 83, Priority:15), (Value: 47, Priority:14), (Value: 22, Priority:15), (Value: 48, Priority:9), (Value: 26, Priority:3), (Value: 44, Priority:3), (Value: 25, Priority:2), (Value: 73, Priority:12), (Value: 41, Priority:7), (Value: 44, Priority:3), (Value: 78, Priority:3), (Value: 61, Priority:2), (Value: 38, Priority:3), (Value: 74, Priority:2), (Value: 20, Priority:1), (Value: 86, Priority:1), (Value: 14, Priority:1), (Value: 7, Priority:2)]
Consumer-3 consumed: (Value: 58, Priority:24) | Buffer: [(Value: 52, Priority:16), (Value: 22, Priority:15), (Value: 83, Priority:15), (Value: 47, Priority:14), (Value: 73, Priority:12), (Value: 48, Priority:9), (Value: 26, Priority:3), (Value: 44, Priority:3), (Value: 25, Priority:2), (Value: 7, Priority:2), (Value: 41, Priority:7), (Value: 44, Priority:3), (Value: 78, Priority:3), (Value: 61, Priority:2), (Value: 38, Priority:3), (Value: 74, Priority:2), (Value: 20, Priority:1), (Value: 86, Priority:1), (Value: 14, Priority:1)]
Consumer-1 consumed: (Value: 52, Priority:16) | Buffer: [(Value: 22, Priority:15), (Value: 47, Priority:14), (Value: 83, Priority:15), (Value: 44, Priority:3), (Value: 73, Priority:12), (Value: 48, Priority:9), (Value: 26, Priority:3), (Value: 74, Priority:2), (Value: 25, Priority:2), (Value: 7, Priority:2), (Value: 41, Priority:7), (Value: 44, Priority:3), (Value: 78, Priority:3), (Value: 61, Priority:2), (Value: 38, Priority:3), (Value: 14, Priority:1), (Value: 20, Priority:1), (Value: 86, Priority:1)]
Consumer-1 consumed: (Value: 22, Priority:15) | Buffer: [(Value: 83, Priority:15), (Value: 47, Priority:14), (Value: 48, Priority:9), (Value: 44, Priority:3), (Value: 73, Priority:12), (Value: 44, Priority:3), (Value: 26, Priority:3), (Value: 74, Priority:2), (Value: 25, Priority:2), (Value: 7, Priority:2), (Value: 41, Priority:7), (Value: 86, Priority:1), (Value: 78, Priority:3), (Value: 61, Priority:2), (Value: 38, Priority:3), (Value: 14, Priority:1), (Value: 20, Priority:1)]
Producer-1 produced: (Value: 3, Priority:94) | Buffer: [(Value: 3, Priority:94), (Value: 83, Priority:15), (Value: 48, Priority:9), (Value: 47, Priority:14), (Value: 73, Priority:12), (Value: 44, Priority:3), (Value: 26, Priority:3), (Value: 74, Priority:2), (Value: 44, Priority:3), (Value: 7, Priority:2), (Value: 41, Priority:7), (Value: 86, Priority:1), (Value: 78, Priority:3), (Value: 61, Priority:2), (Value: 38, Priority:3), (Value: 14, Priority:1), (Value: 20, Priority:1), (Value: 25, Priority:2)]
Producer-2 produced: (Value: 66, Priority:40) | Buffer: [(Value: 3, Priority:94), (Value: 66, Priority:40), (Value: 48, Priority:9), (Value: 83, Priority:15), (Value: 73, Priority:12), (Value: 44, Priority:3), (Value: 26, Priority:3), (Value: 74, Priority:2), (Value: 47, Priority:14), (Value: 7, Priority:2), (Value: 41, Priority:7), (Value: 86, Priority:1), (Value: 78, Priority:3), (Value: 61, Priority:2), (Value: 38, Priority:3), (Value: 14, Priority:1), (Value: 20, Priority:1), (Value: 25, Priority:2)]
Consumer-1 consumed: (Value: 3, Priority:94) | Buffer: [(Value: 66, Priority:40), (Value: 83, Priority:15), (Value: 48, Priority:9), (Value: 47, Priority:14), (Value: 73, Priority:12), (Value: 44, Priority:3), (Value: 26, Priority:3), (Value: 74, Priority:2), (Value: 44, Priority:3), (Value: 7, Priority:2), (Value: 41, Priority:7), (Value: 86, Priority:1), (Value: 78, Priority:3), (Value: 61, Priority:2), (Value: 38, Priority:3), (Value: 14, Priority:1), (Value: 20, Priority:1)]
Consumer-2 consumed: (Value: 66, Priority:40) | Buffer: [(Value: 83, Priority:15), (Value: 47, Priority:14), (Value: 48, Priority:9), (Value: 44, Priority:3), (Value: 73, Priority:12), (Value: 44, Priority:3), (Value: 26, Priority:3), (Value: 74, Priority:2), (Value: 25, Priority:2), (Value: 7, Priority:2), (Value: 41, Priority:7), (Value: 86, Priority:1), (Value: 78, Priority:3), (Value: 61, Priority:2), (Value: 38, Priority:3), (Value: 14, Priority:1), (Value: 20, Priority:1)]
----- STATISTICS -----
Producer-1 produced: 305 items
Producer-2 produced: 281 items
Consumer-1 consumed: 199 items
Consumer-2 consumed: 185 items
Consumer-3 consumed: 185 items
Remaining items in buffer: 17

```

Task 3: Thread Pool Web Crawler Simulation

Create a web crawler simulation using thread pools:

- Simulate downloading web pages (use Thread.sleep() for delay)
- Each "page" has a random download time (1-5 seconds)
- Use different thread pool types: FixedThreadPool, CachedThreadPool, SingleThreadExecutor
- Process 20 URLs and compare performance

Requirements:

- Measure total execution time for each thread pool type
- Handle timeouts (max 10 seconds per page)
- Display which thread processed each URL

Step execution and approach:

- This task run in Task3.java
- Simulate a web crawler by treating each URL as a task (DownloadPage) that pretends to download a page using Thread.sleep() with a random delay.
- Use Callable instead of Runnable so each task can return a result indicating which thread processed which URL and how long it took.
- Submit all URL tasks to an ExecutorService (thread pool), which manages thread creation, reuse, and task scheduling automatically.
- Store returned Future objects to:
 - Wait for task completion
 - Retrieve results
 - Enforce a maximum timeout of 10 seconds per page
- Run the same set of 20 URLs using three different thread pool types (Fixed, Cached, Single) to compare parallelism and total execution time.
- Measure performance by recording start and end time for each pool and printing which thread processed each URL.

Output:
Running with FixedThreadPool

```
pool-1-thread-1 processed page1 in 2 ms
pool-1-thread-2 processed page2 in 4 ms
pool-1-thread-3 processed page3 in 1 ms
pool-1-thread-4 processed page4 in 5 ms
pool-1-thread-5 processed page5 in 2 ms
pool-1-thread-3 processed page6 in 4 ms
pool-1-thread-5 processed page7 in 1 ms
pool-1-thread-1 processed page8 in 4 ms
pool-1-thread-5 processed page9 in 4 ms
pool-1-thread-2 processed page10 in 4 ms
pool-1-thread-4 processed page11 in 1 ms
pool-1-thread-3 processed page12 in 5 ms
pool-1-thread-4 processed page13 in 5 ms
pool-1-thread-1 processed page14 in 2 ms
pool-1-thread-5 processed page15 in 3 ms
pool-1-thread-2 processed page16 in 5 ms
pool-1-thread-1 processed page17 in 3 ms
pool-1-thread-5 processed page18 in 5 ms
pool-1-thread-3 processed page19 in 3 ms
pool-1-thread-4 processed page20 in 3 ms
Total execution time: 15048 ms
```

Running with CachedThreadPool

```
pool-2-thread-1 processed page1 in 3 ms
pool-2-thread-2 processed page2 in 5 ms
pool-2-thread-3 processed page3 in 5 ms
pool-2-thread-4 processed page4 in 2 ms
pool-2-thread-5 processed page5 in 1 ms
pool-2-thread-6 processed page6 in 1 ms
pool-2-thread-7 processed page7 in 3 ms
pool-2-thread-8 processed page8 in 5 ms
pool-2-thread-9 processed page9 in 2 ms
pool-2-thread-10 processed page10 in 4 ms
pool-2-thread-11 processed page11 in 4 ms
pool-2-thread-12 processed page12 in 3 ms
pool-2-thread-13 processed page13 in 4 ms
pool-2-thread-14 processed page14 in 4 ms
pool-2-thread-15 processed page15 in 1 ms
pool-2-thread-16 processed page16 in 4 ms
pool-2-thread-17 processed page17 in 5 ms
pool-2-thread-18 processed page18 in 1 ms
pool-2-thread-19 processed page19 in 5 ms
pool-2-thread-20 processed page20 in 5 ms
Total execution time: 5013 ms
```

Running with SingleThreadExecutor

```
pool-3-thread-1 processed page1 in 2 ms
pool-3-thread-1 processed page2 in 1 ms
pool-3-thread-1 processed page3 in 4 ms
pool-3-thread-1 processed page4 in 2 ms
pool-3-thread-1 processed page5 in 2 ms
pool-3-thread-1 processed page6 in 5 ms
pool-3-thread-1 processed page7 in 4 ms
pool-3-thread-1 processed page8 in 1 ms
pool-3-thread-1 processed page9 in 3 ms
pool-3-thread-1 processed page10 in 1 ms
```

```
pool-3-thread-1 processed page11 in 5 ms
pool-3-thread-1 processed page12 in 3 ms
pool-3-thread-1 processed page13 in 3 ms
pool-3-thread-1 processed page14 in 2 ms
pool-3-thread-1 processed page15 in 3 ms
pool-3-thread-1 processed page16 in 4 ms
pool-3-thread-1 processed page17 in 2 ms
pool-3-thread-1 processed page18 in 1 ms
pool-3-thread-1 processed page19 in 1 ms
pool-3-thread-1 processed page20 in 5 ms
Total execution time: 54165 ms
```

Task 4: Bank Account Race Condition Demonstration

Create a program that demonstrates race conditions and their solutions:

- Implement a BankAccount class with deposit/withdraw methods
- Create both synchronized and non-synchronized versions
- Run 10 threads performing 100 random transactions each
- Compare final balances between synchronized and non-synchronized versions

Requirements:

- Start with initial balance of \$1000
- Each transaction should be \$1-50 (random)
- Display transaction history and final balance
- Explain the difference in results

Step execution and approach:

- Running your code in Task4.java which contain the logic in BankAccount.java
- Create a shared BankAccount object with initial balance \$1000.
- Define deposit and withdraw methods in both synchronized and non-synchronized versions.
- Create 10 threads, each performing 100 random transactions (deposit or withdraw, \$1-\$50).
- Use a shared StringBuffer to record transaction history.
- Start all threads and wait for them to finish using join().
- Compare results: non-synchronized version may show incorrect final balance due to race conditions, synchronized version ensures correct and consistent balance.\
- Display transaction history and final balance for both versions.

Output:

```
Thread-5 withdraw $1 | balance: 415
Thread-7 deposits $46 | balance: 461
Thread-5 withdraw $44 | balance: 417
Thread-7 deposits $18 | balance: 435
Thread-5 withdraw $38 | balance: 397
Thread-7 deposits $36 | balance: 433
Thread-5 deposits $25 | balance: 458
Thread-7 deposits $5 | balance: 463
Thread-5 withdraw $31 | balance: 432
Thread-7 withdraw $32 | balance: 400
Thread-7 withdraw $41 | balance: 359
Thread-7 withdraw $21 | balance: 338
Thread-7 deposits $16 | balance: 354
Thread-7 deposits $14 | balance: 368

Final balance: $368
```

```
Thread-10 deposits $43 | balance: 1386
Thread-10 deposits $48 | balance: 1434
Thread-10 deposits $3 | balance: 1437
Thread-10 withdraw $44 | balance: 1393
Thread-10 withdraw $27 | balance: 1366
Thread-10 deposits $34 | balance: 1400
Thread-10 withdraw $1 | balance: 1399
Thread-10 withdraw $23 | balance: 1376
Thread-10 withdraw $47 | balance: 1329
Thread-10 withdraw $18 | balance: 1311
Thread-10 withdraw $49 | balance: 1262
Thread-10 deposits $27 | balance: 1289
Thread-10 withdraw $2 | balance: 1287
Thread-10 withdraw $44 | balance: 1243
Thread-10 deposits $5 | balance: 1248
Thread-10 withdraw $31 | balance: 1217
Thread-10 deposits $49 | balance: 1266
Thread-10 withdraw $23 | balance: 1243
Thread-10 deposits $22 | balance: 1265
Thread-10 withdraw $25 | balance: 1240
Thread-10 deposits $38 | balance: 1278

Final balance: $1278
```